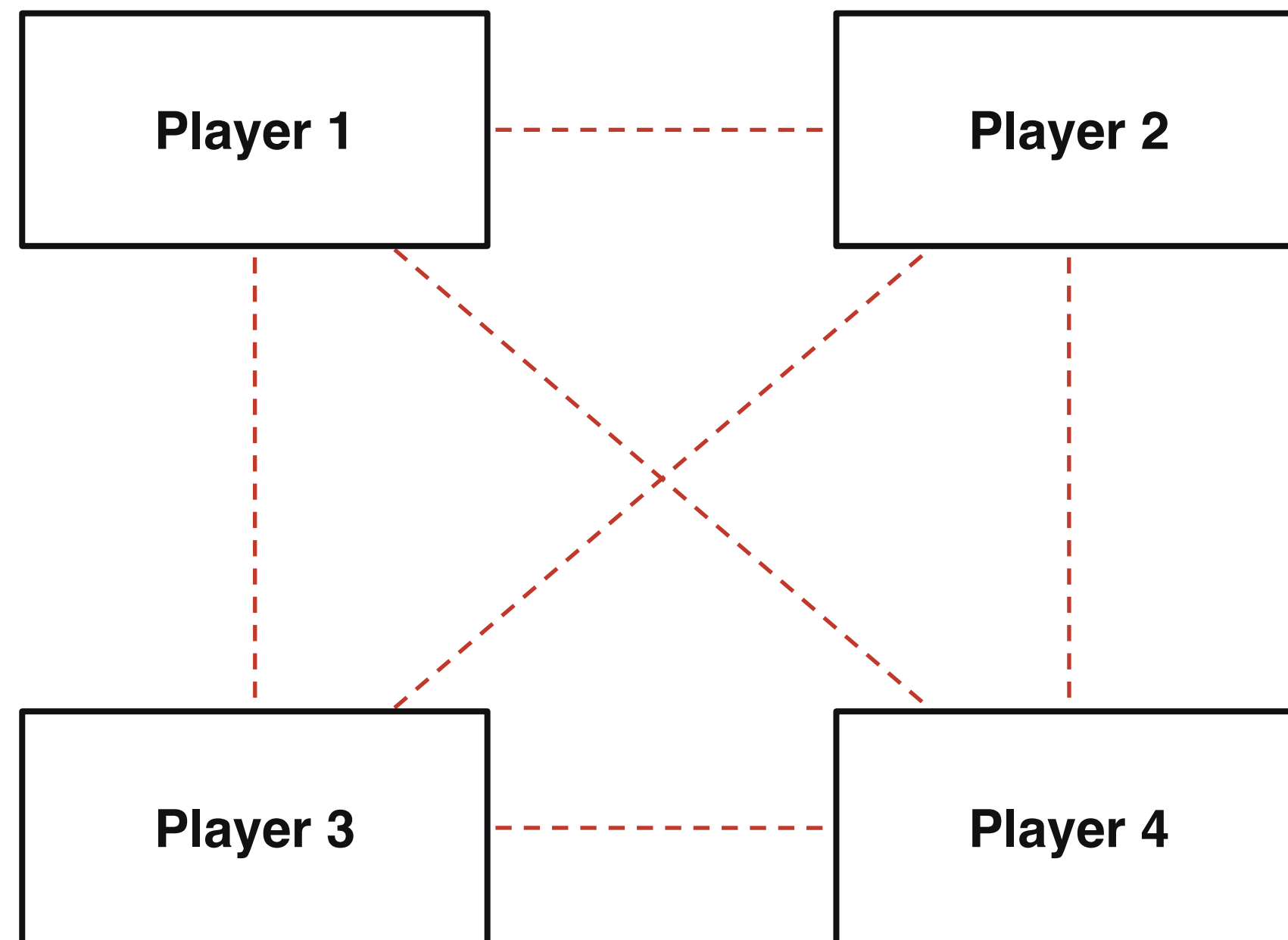


Building Multiplayer Experiments with Humans and LLM Agents

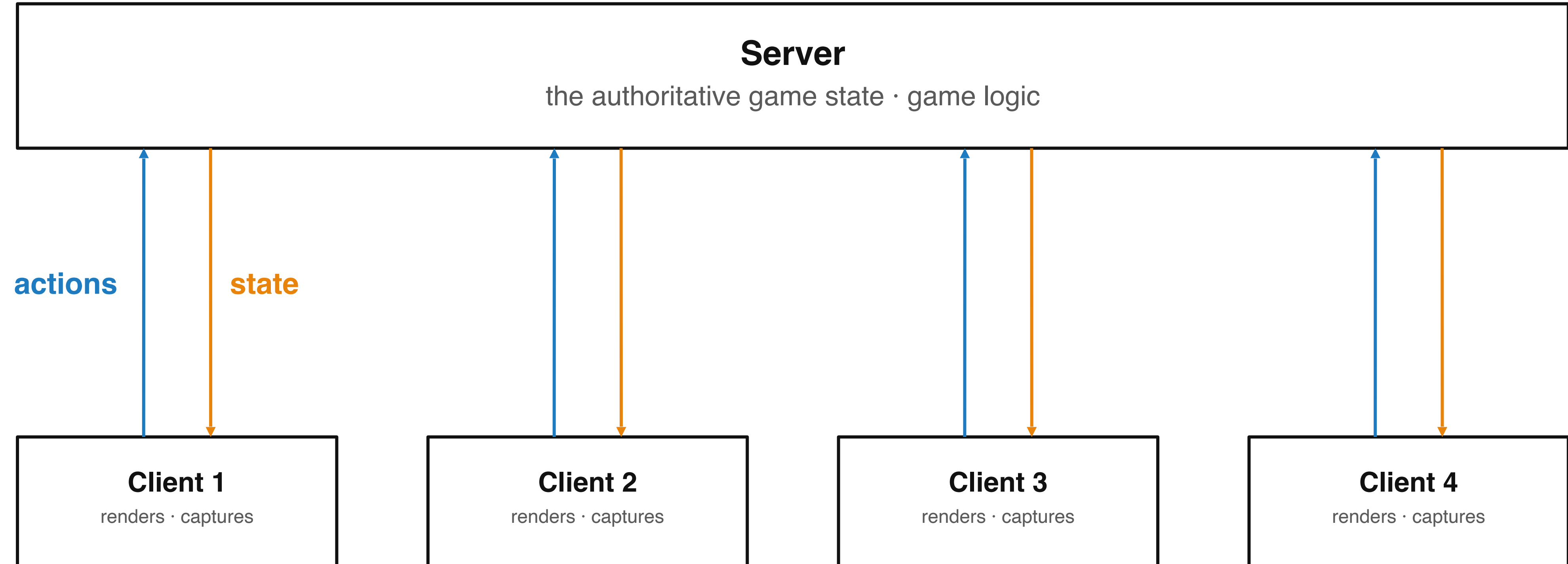
A hands-on walkthrough with Empirica, Multi-LLM and our hiring game

The technical challenge: synchronization



- **Every action reaches every player** — in real time, mid-experiment.
- **All views agree** — same counts, same round, same timer.
- **Advance in lockstep** — only when the whole group is ready.

Multi-human: a server–client architecture



What is Empirica?

Empirica is a free, open-source, virtual lab platform for developing and conducting synchronous and interactive human-participant experiments.

Behavior Research Methods (2021) 53:2158–2171
<https://doi.org/10.3758/s13428-020-01535-9>



Empirica: a virtual lab for high-throughput macro-level experiments

Abdullah Almaatouq¹  · Joshua Becker²  · James P. Houghton³ · Nicolas Paton¹ · Duncan J. Watts³  · Mark E. Whiting³ 

Accepted: 30 December 2020 / Published online: 29 March 2021
© The Author(s) 2021



<https://empirica.ly/>

Try it first



`http://184.73.9.243:3000/`

join, play one round

Hands-on: begin your own TogetherHire

Follow along — all commands are in the README: github.com/jouisseuse/IC2S2-26-Tutorial

- 1 Install prerequisites
- 2 Install Empirica
- 3 Clone the repo
- 4 Run the app
- 5 Open player + admin tabs
- 6 Create a batch — and play

Setup ① Prerequisites

```
# install, then confirm versions:
```

```
$ git --version  
$ node --version  
$ npm --version
```

```
# Windows: use WSL
```

```
PS> wsl --install
```

- **WSL tip:** enable Ctrl+Shift+C/V copy-paste in terminal Properties → Options.

Setup ② Install Empirica

```
# install the Empirica CLI (also runs `empirica setup`)  
$ curl -fsS https://install.empirica.dev | sh  
$ empirica version  
✓ a version number = ready
```

```
# (NOT for today) create new project & run  
$ empirica create PJNAME # scaffolds a runnable demo app  
$ cd PJNAME && empirica
```

Setup ③ Install TogetherHire

```
$ git clone https://github.com/jouisseuse/IC2S2-26-Tutorial.git
$ cd IC2S2-26-Tutorial
$ bash setup_togetherhire.sh
```

- **The script does:** checks env → creates ../my-experiment → copies files → installs deps → startup test.

```
# options:
$ bash setup_togetherhire.sh ../my-togetherhire-test    # different folder
$ RUN_STARTUP_TEST=0 bash setup_togetherhire.sh         # skip the test
```

Setup ④ Run TogetherHire

```
$ cd ../my-experiment
```

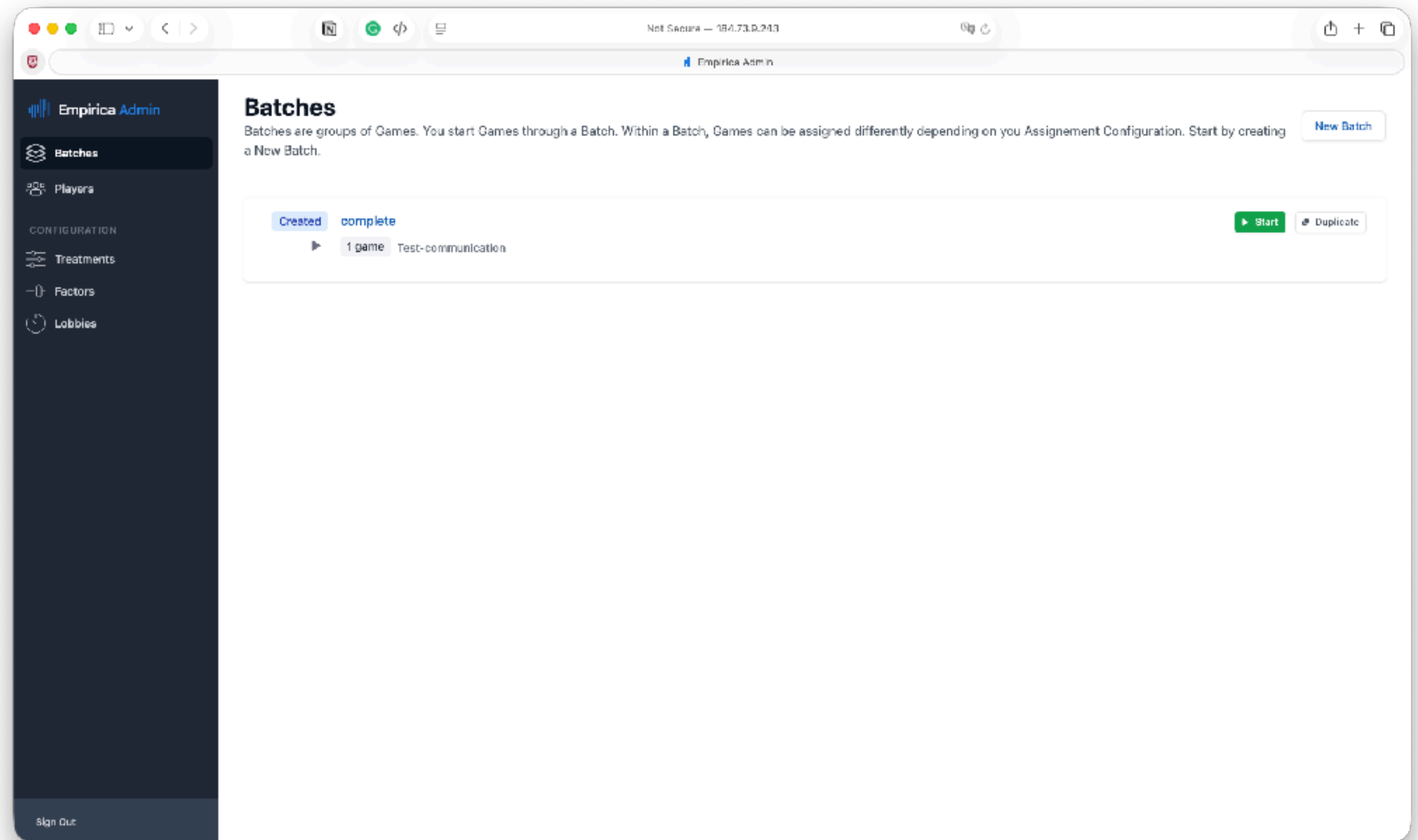
```
$ empirica
```

✓ prints local **Player** and **Admin** URLs — open both

Setup ⑤ Run a batch

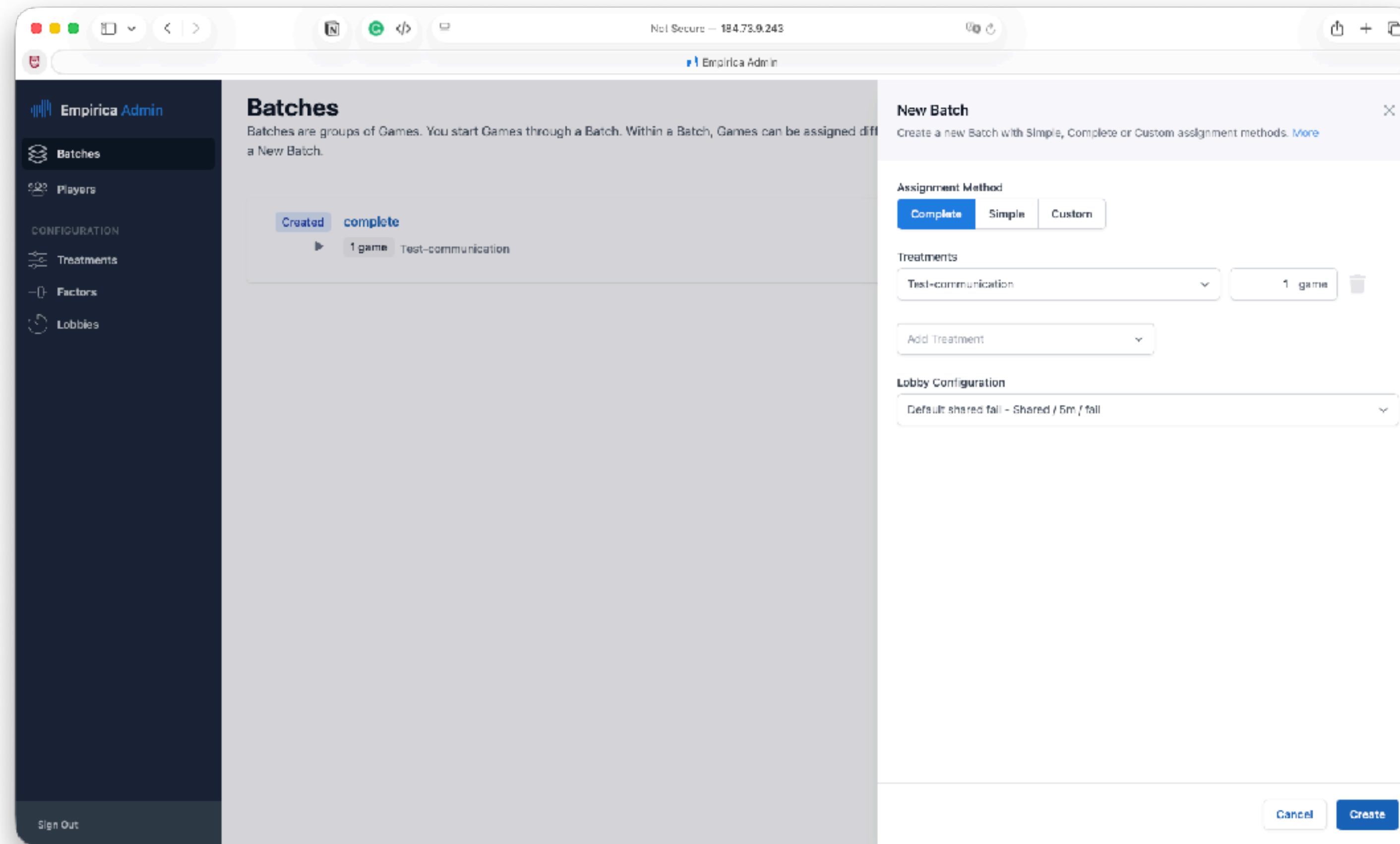
```
# 6 • run TogetherHire  
$ empirica  
→ open the participant + admin  
   links it prints  
  
# quick local test:  
# open TWO participant windows  
# so a group can form
```

1 localhost:3000/admin



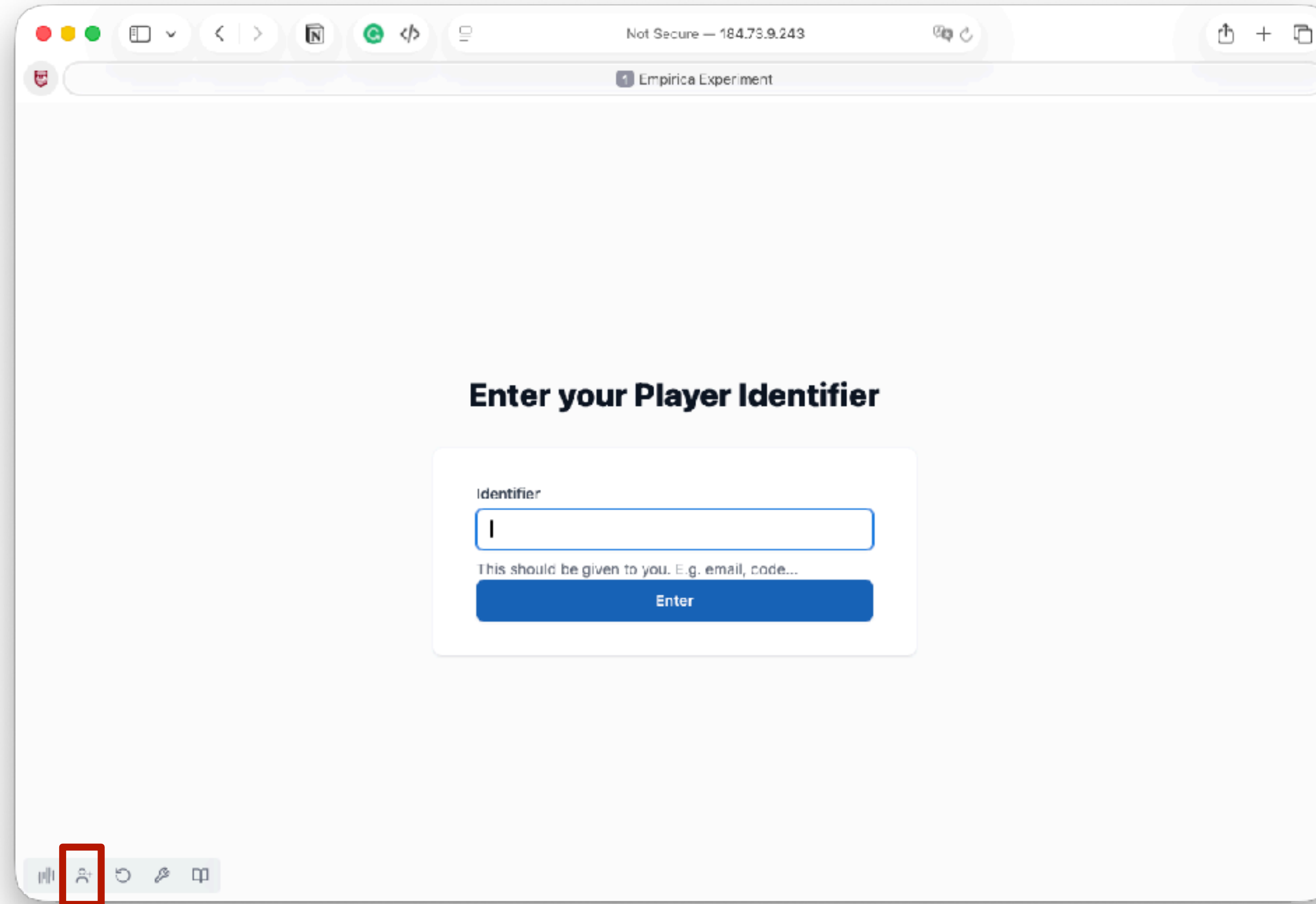
Setup ⑤ Run a batch

2 New Batch → pick a treatment → Start



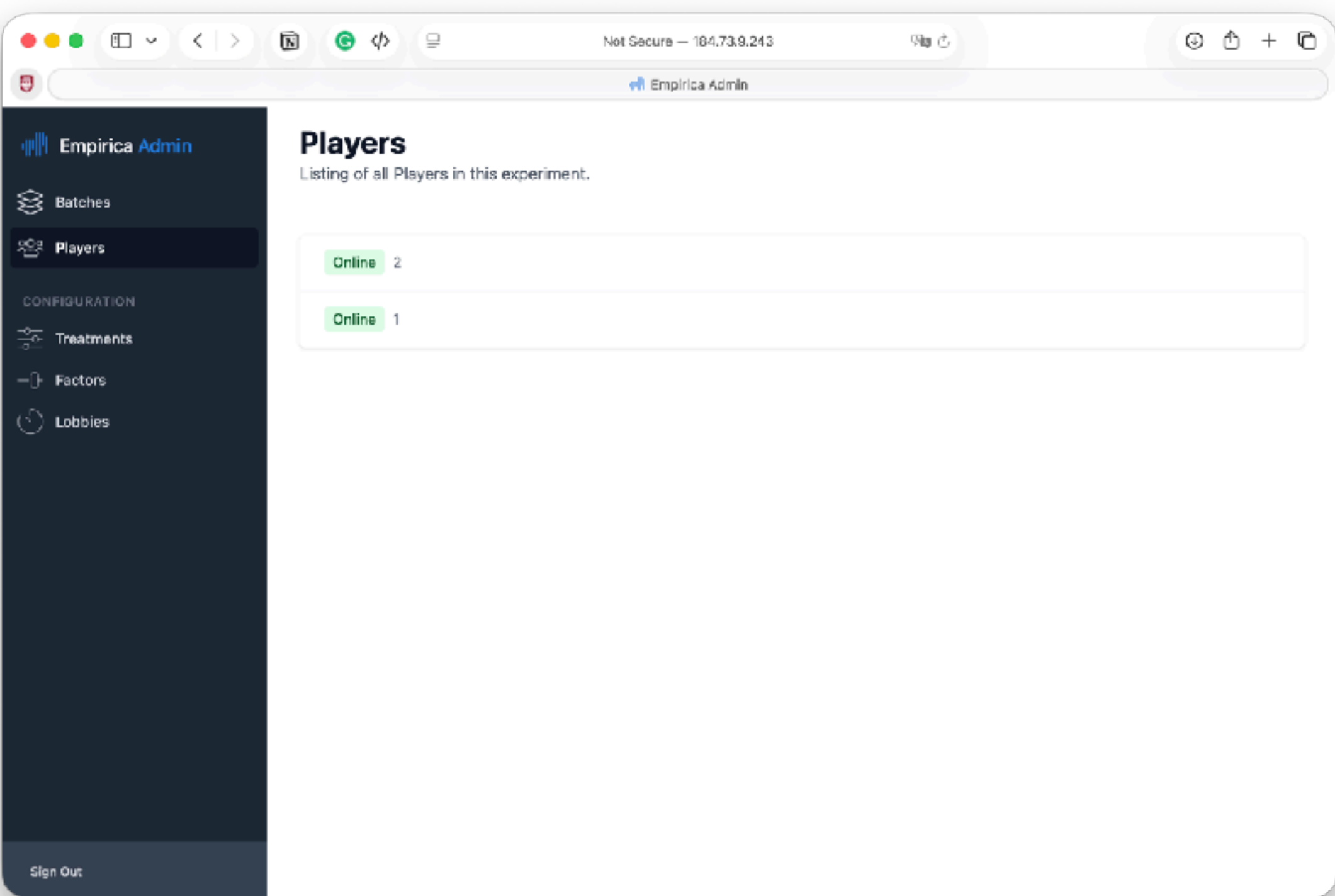
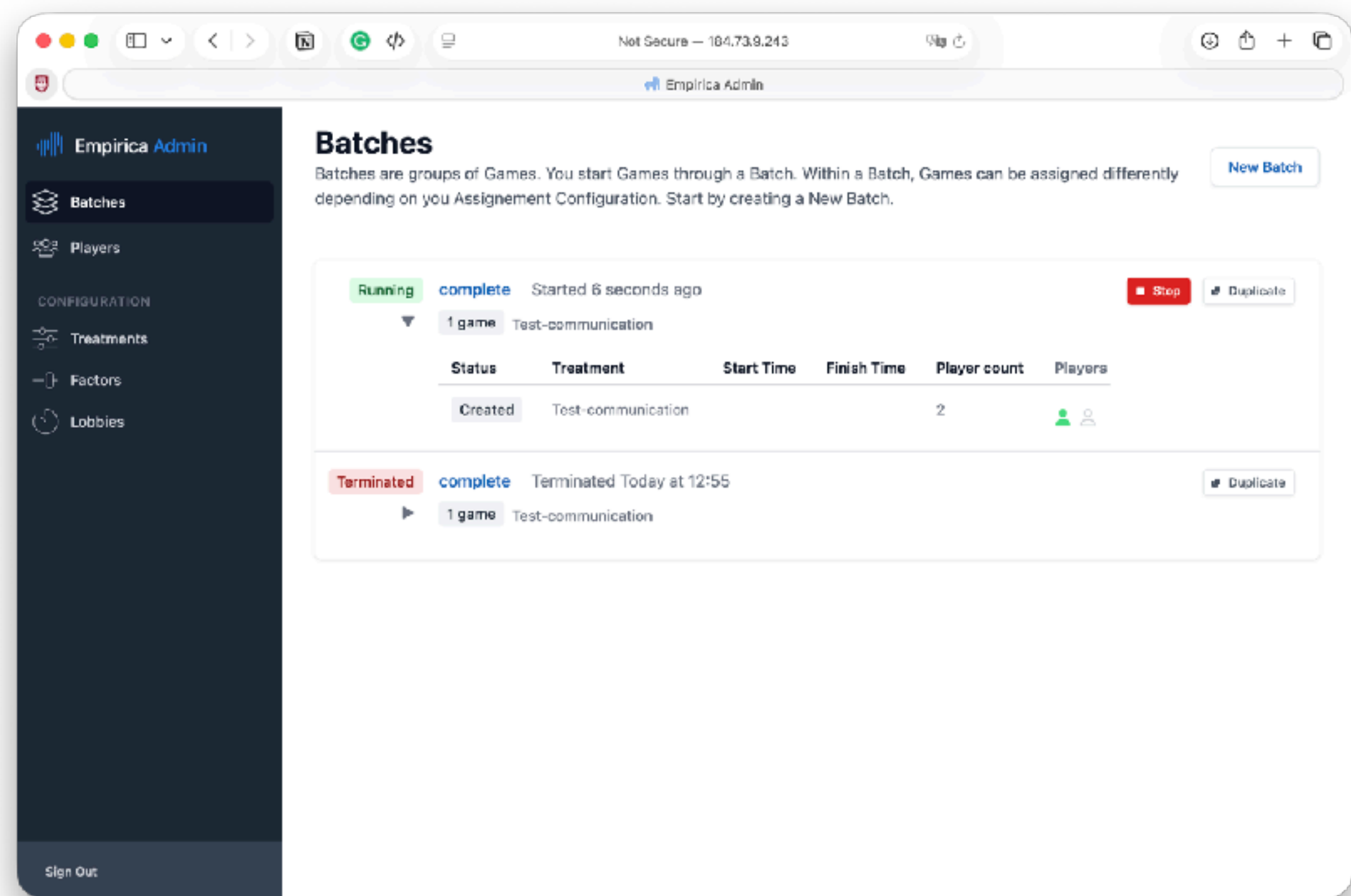
Setup ⑤ Run a batch

3 localhost:3000



4 Open more player tabs

Monitor the game through the admin panel



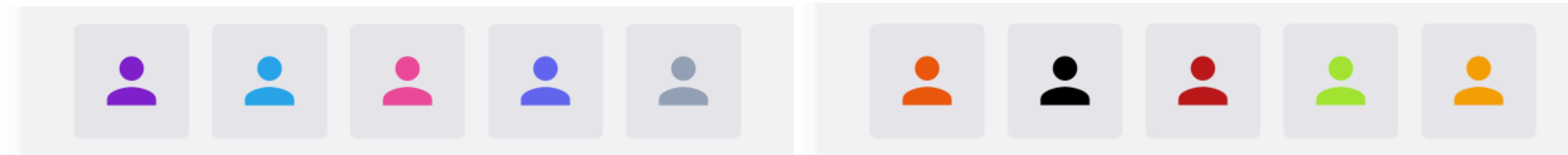
What's in the folder

```
TogetherHire/
├── .empirica/
│   └── treatments.yaml           conditions
├── server/src/
│   └── callbacks.js             all game logic
└── client/src/
    ├── App.jsx                   wiring: consent · lobby · intro · exit
    ├── Game.jsx                  layout: Profile + Stage
    ├── Stage.jsx                 routes stage name → component
    ├── Profile.jsx               top bar: round · timer · bonus
    ├── stages/
    │   ├── Choice.jsx            the game screen
    │   ├── GroupAllocation.jsx    end-of-game task
    │   ├── GameIntroduction.jsx   per-block instructions
    │   └── Result.jsx            (unused)
    ├── intro-exit/
    │   ├── MyConsent.jsx         IRB consent
    │   ├── Introduction.jsx      welcome screen
    │   └── ExitSurvey.jsx        3-page survey
    ├── components/
    │   ├── MyLobby.jsx           custom lobby (trivia)
    │   ├── LoadingOverlay.jsx    connecting / loading
    │   └── Timer · Button · Avatar · Alert · ...
```

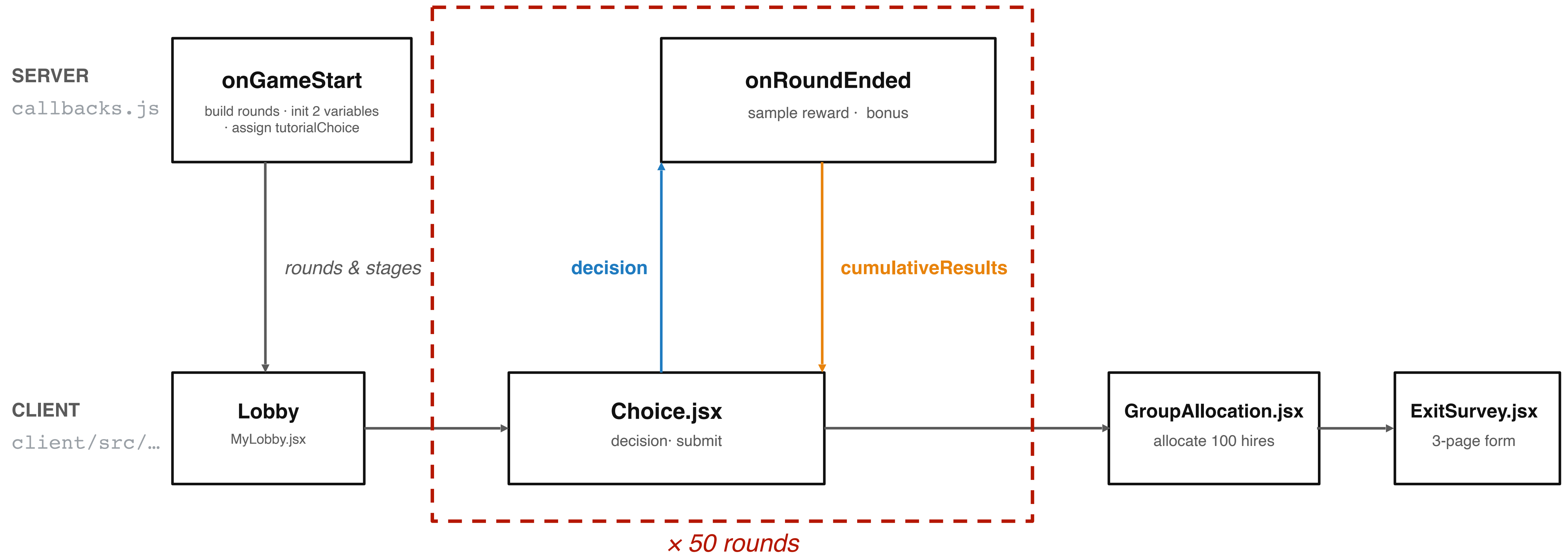
- **callbacks.js** = what happens.
- **stages/** = what players see.
- **treatments.yaml** = the conditions.

Our game: hiring as a multi-armed bandit

- **Task.** Each round, hire one intern from one of 10 groups. It succeeds or fails.
- **Hidden truth.** Each group has a fixed success rate (0.10 – 0.95).
- **Incentive.** 50 rounds; +1¢ per success. Explore or exploit.
- **Manipulation.** Groups of 10 — with or without seeing each other's results.



server/src/callbacks.js — the game, one picture



server/src/callbacks.js — onGameStart

```
Empirica.onGameStart(({ game }) => {  
  // two ledgers, Laplace prior 1/1  
  game.set("cumulativeResults", initial);  
  game.players.forEach((p, i) => {  
    p.set("cumulativeResults", initial);  
    p.set("tutorialChoice", options[i % 10]);  
  });  
  
  // the rounds ARE the client's flow:  
  addIntroductionRound(game); // → GameIntroduction.jsx  
  addTutorialRound(game);     // → Choice.jsx (60 s)  
  addRounds(game, 50, chatEnabled); // → Choice.jsx (30 s)  
  addPostGameSurvey(game);     // → GroupAllocation.jsx  
});
```

- **A round's stage maps client screen** —
“choice” → Choice.jsx,
“Group-Allocation” → GroupAllocation.jsx.
- **Two variables born here:** game-level (group)
and player-level (private).
- **tutorialChoice:** player i → arm i — all 10 arms
sampled once.

server/src/callbacks.js — the two conditions

```
// server: condition stamped on every round
game.addRound({
  name: `${chatEnabled ? "Communication"
    : "Non-Communication"} Round ${i+1}`,
  chatEnabled, flag: true,
});
round.addStage({ name: "choice", duration: 30 });
```

- **The manipulation = which variables the client reads.**

```
// "combined": within-subject
// type === "combined": asocial block, then social block
addRounds(game, n, false);
addRounds(game, n, true);
```

- **Condition rides on the round → blocks can alternate.**

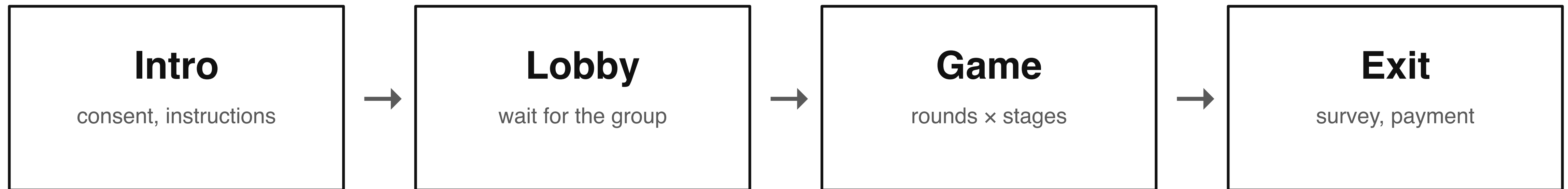
server/src/callbacks.js — onRoundEnded

```
// the truth lives HERE — server-side only
const CANDIDATE_PROBABILITIES = { "Bright Green": 0.95,
  Slate: 0.8, "Sky Blue": 0.68, Crimson: 0.53, ..., Purple: 0.1 };
const sampleReward = (c) =>
  Math.random() < CANDIDATE_PROBABILITIES[c] ? 1 : 0;

Empirica.onRoundEnded(({ round }) => {
  if (round.get("flag") !== true) return;
  for (const player of players) {
    const choice = player.round.get("decision"); // ← Choice.jsx
    let score = player.round.get("score");
    if (typeof score !== "number") {
      score = sampleReward(choice); // reward drawn HERE
      player.round.set("score", score);
    }
    bump(gameCum[choice], score); bump(playerCum[choice], score);
    player.set("cumulativeResults", playerCum); // → asocial display
    player.set("score", total + score); // bonus (+1¢)
  }
  game.set("cumulativeResults", gameCum); // → social display
});
```

- **The reward draw is server-side now** — the truth table never reaches the browser.
- ← **blue:** reads the client's decision.
- → **orange:** writes next round's display.

A participant's lifecycle



Empirica manages the transitions. You define what happens inside each phase.

client/src/App.jsx — the client in one file

```
<EmpiricaContext
  consent={MyConsent}           // IRB
  introSteps={ [Introduction]}  // "DO NOT Exit"
  lobby={MyLobby}               // → ops section
  exitSteps={ [ExitSurvey]}>
  <Game />                      // Profile + Stage
</EmpiricaContext>

// Stage.jsx — stage name → screen:
switch (stage.get("name")) {
  case "introduction": return <GameIntroduction/>;
  case "choice":       return <Choice/>;
  case "Group-Allocation": return <GroupAllocation/>;
}
```

- **Every phase is a prop.** Swap any screen by passing a component.
- **Stage.jsx completes the map** from onGameStart's stage names to screens.

client/src/stages/Choice.jsx — the game screen



- **5x2 grid** of person icons — one per university.
- **Success / Failure counts** under each button — fed by cumulativeResults.
- **Timer warning** in the last 10 s.
- **Seen 50 times** per participant.



client/src/stages/Choice.jsx — click + condition

```
function onClick(candidate) {  
  player.round.set("decision", candidate.name);  
  player.stage.set("submit", true);  
}
```

- **Three writes up** — the arrows from the pipeline.
- **One ternary** = the whole communication manipulation.

```
// THE experiment — which ledger feeds the display:  
const cum = round.get("chatEnabled")  
  ? game.get("cumulativeResults")    // social: pooled  
  : player.get("cumulativeResults"); // asocial: own
```

The rest of the client components

client/src/stages/GroupAllocation.jsx

allocate exactly 100 hires; submit locks until 0 remain → groupAllocations

client/src/intro-exit/ExitSurvey.jsx

3-page validated form → exitSurvey (one JSON write)

client/src/intro-exit/MyConsent.jsx · Introduction.jsx

IRB consent

tutorial round (callbacks.js + Choice.jsx)

assigned arms sample all 10 once; isTutorial → dropped in analysis

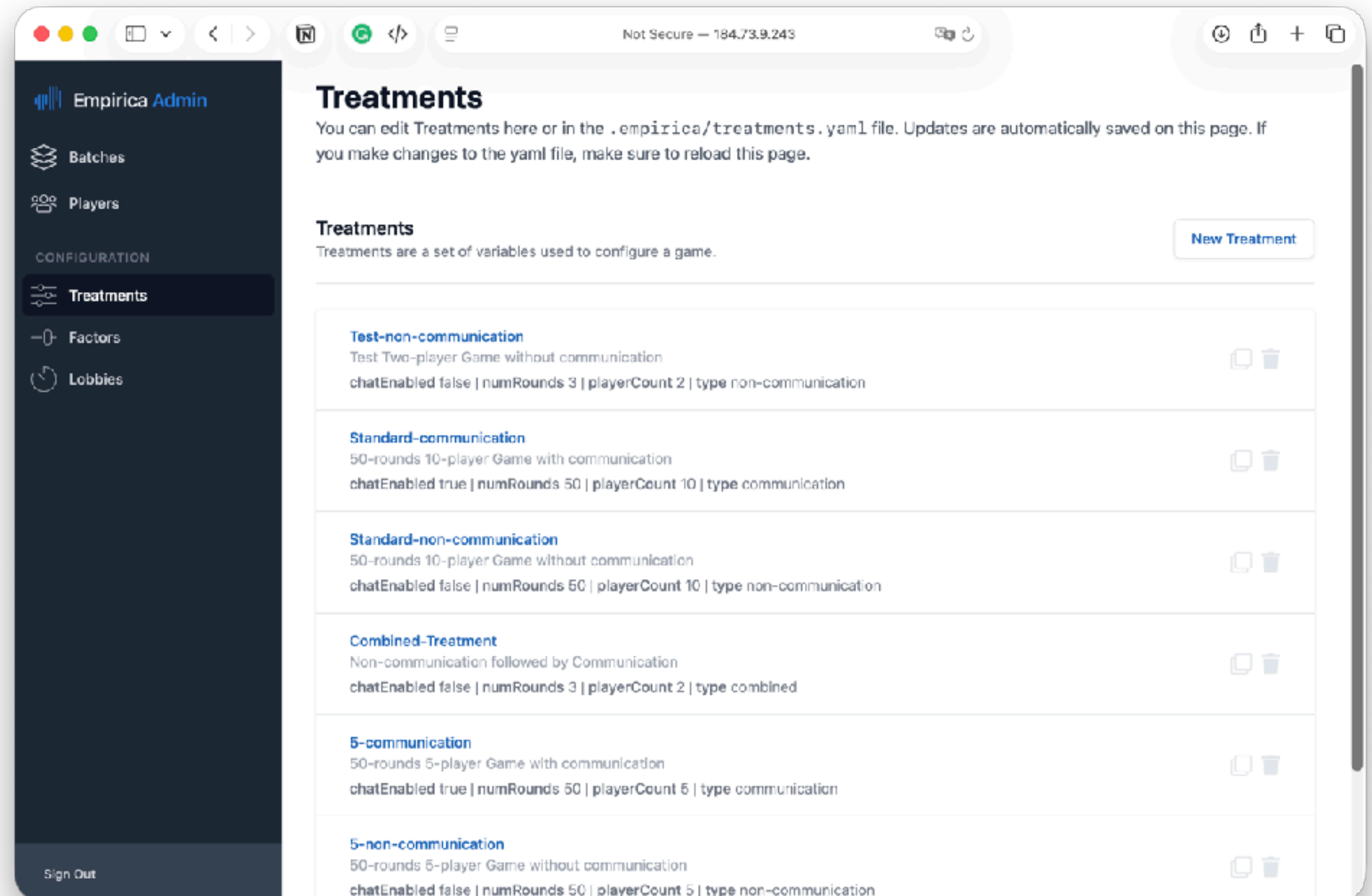
client/src/components/

Timer · Button · Avatar · LoadingOverlay · MyLobby (→ ops)

.empirica/treatments.yaml: conditions as configuration

```
# .empirica/treatments.yaml
factors:
  - name: playerCount
    values: [{ value: 10 }]
  - name: numRounds
    values: [{ value: 50 }]
  - name: type
    values:
      - value: communication
      - value: non-communication
      - value: combined          # within-subject

treatments:
  - name: Standard-non-communication
    factors:
      playerCount: 10
      numRounds: 50
      type: non-communication
```



`empirica export` — turns tajriba.json into CSVs

```
$ empirica export
INF Starting CSV export...
  output = hiring-game-....zip

INF Exporting batch      count=3  keys=5
INF Exporting game      count=3  keys=13
INF Exporting global     count=1  keys=1
INF Exporting player     count=3  keys=9
INF Exporting playerGame count=2  keys=3
INF Exporting playerRound count=4  keys=6
INF Exporting playerStage count=6  keys=6
INF Exporting round      count=2  keys=10
INF Exporting stage      count=3  keys=11
```

- **One timestamped .zip**, nine CSVs — one per entity type.
- **count** = rows · **keys** = columns.
- **Runs anytime** — doesn't stop live batches.
- **Loads straight into pandas / R.**

One participant, traced through the real data

```
playerRound.csv    decision=Crimson    score=1.0          (round after round...)

player.csv         cumulativeResults = { "Crimson": { success: 51, failures: 2 },
                      "Amber": { success: 1, failures: 1 }, ... }

player.csv         groupAllocations   = { "Crimson": 100 }          entropy = 0

player.csv         treatmentName      = Standard-non-communication
```

- **The story the data tells:** explored briefly, found Crimson, exploited it — 50 wins (51 incl. the 1/1 Laplace prior) — and allocated all 100 final hires to it: allocation entropy zero.
- **Behavior (playerRound) + beliefs (allocations) + condition (treatment)** join on player and game IDs.

Four strategies to solve the empty-room problem

1. Scheduled sessions

Recruit for a fixed time slot, not “join anytime.” The single biggest win for synchronization.

2. Over-recruitment

No-shows are normal. Need 10 players? Invite 15–20. A half-empty session costs the whole session.

3. Improve the lobby

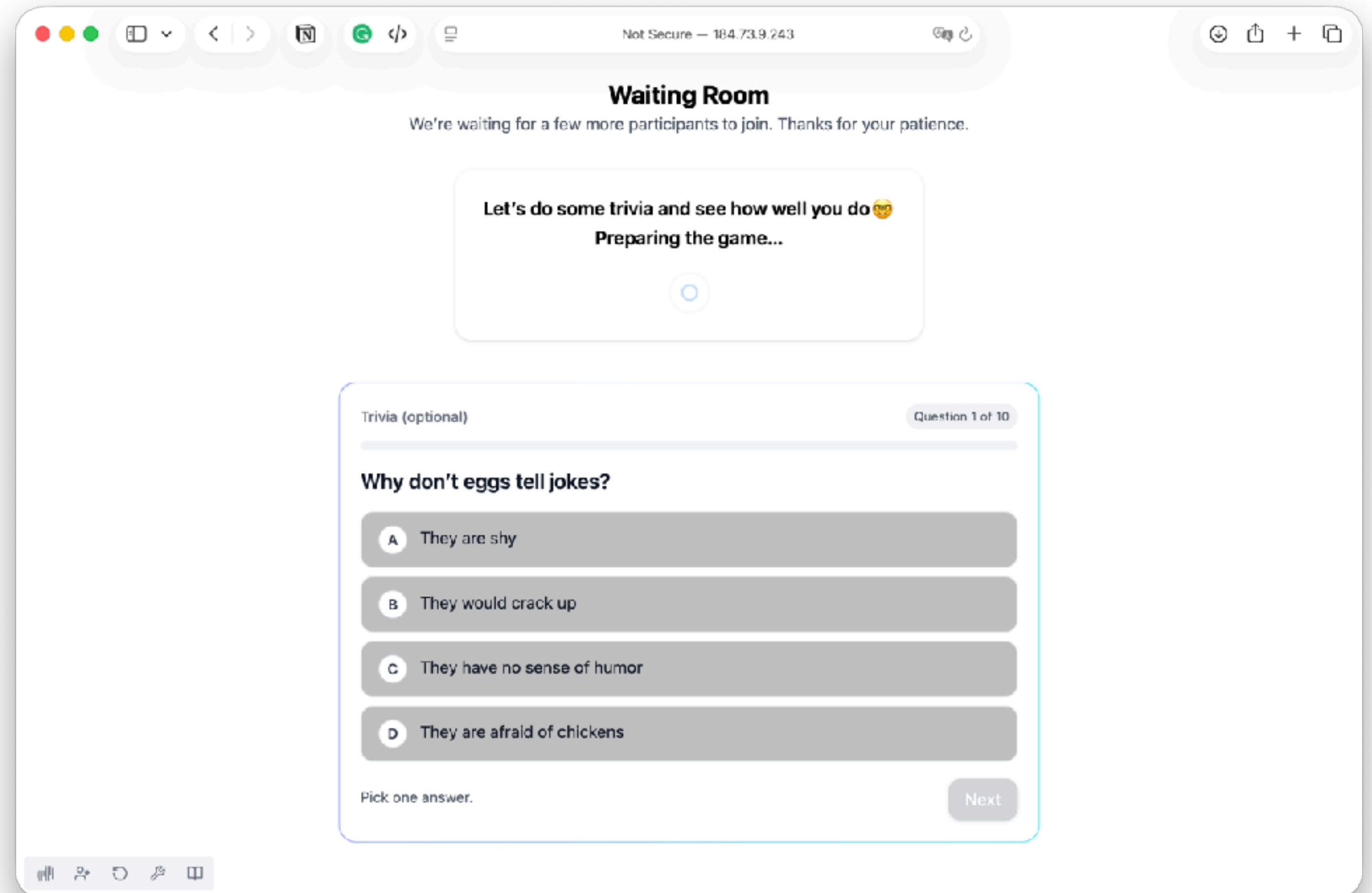
Pay for waiting time. Show a countdown. Notify at start. Ours: the trivia lobby from MyLobby.jsx.

4. Smaller groups

Groups fill much faster and are operationally more stable. Keep N as small as the design allows.

MyLobby.jsx — the waiting room

- **Replaces the default spinner** with a proper waiting room.
- **Optional joke trivia** — 9 questions, progress bar, score — keeps players at the tab.
- **ETA** from `treatment.lobbyDurationSec`.



The other things Empirica supports

RESEARCH ARTICLE | SOCIAL SCIENCES | 



Adaptive social networks promote the wisdom of crowds

Abdullah Almaatouq  , Alejandro Noriega-Campero, Abdulrahman Alotaibi                                                      

The other things Empirica supports

RESEARCH ARTICLE | PSYCHOLOGICAL AND COGNITIVE SCIENCES |

Interaction structure constrains the emergence of conventions in group communication

Veronica Boyce , Robert D. Hawkins, Noah D. Goodman, and Michael C. Frank [Authors Info & Affiliations](#)

Edited by Gary Dell, University of Illinois at Urbana-Champaign, Champaign, IL; received February 26, 2024; accepted June 5, 2024

July 5, 2024 | 121 (28) e2403888121 | <https://doi.org/10.1073/pnas.2403888121>

2,036 | 7

PDF EPUB

Open access | | Research article | First published online April 18, 2024 | [Re-use permissions](#)

Algorithmically mediating communication to enhance collective decision-making in online social networks

Jason W Burton , Abdullah Almeatoug , ..., and Ulrike Hahn [View all authors and affiliations](#)

[All Articles](#) | <https://doi.org/10.1177/26339137241241307>

Contents | PDF/EPUB Cite Share options Information, rights and permissions Metrics and citations



Real-time communication, group deliberation

The other things Empirica supports

[nature](#) > [nature human behaviour](#) > [articles](#) > article

Article | [Open access](#) | Published: 19 May 2025

On the conversational persuasiveness of GPT-4

[Francesco Salvi](#) , [Manoel Horta Ribeiro](#), [Riccardo Gallotti](#) & [Robert West](#)

[Nature Human Behaviour](#) **9**, 1645–1653 (2025) | [Cite this article](#)

100k Accesses | **101** Citations | **1811** Altmetric | [Metrics](#)

Incorporate agents, e.g. LLM agent

Hybrid social learning in human-algorithm cultural transmission

[L Brinkmann](#)^{1,∞}, [D Gezerli](#)¹, [K V Kleist](#)¹, [T F Müller](#)¹, [I Rahwan](#)¹, [N Pescetelli](#)^{1,2}

► [Author information](#) ► [Article notes](#) ► [Copyright and License information](#)

PMCID: PMC9126184 PMID: [35599570](#)

Cultural transmission, network evolution

For more, please refer Empirica docs: `docs.empirica.ly`

Part 2 — The Same Game, Played by LLMs

Multi-agent bandit simulation with GPT agents · [LLM/mamab_llm.py](#)

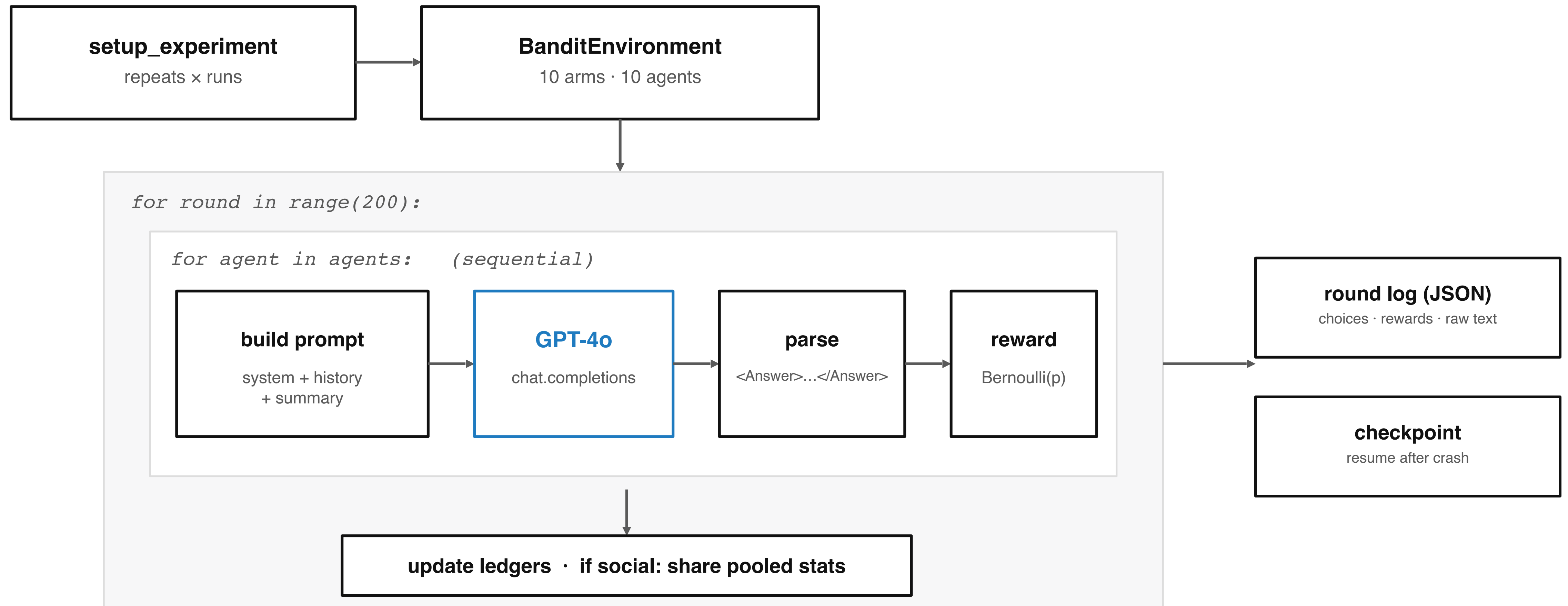
The LLM API (GPT)

```
from openai import OpenAI
client = OpenAI(api_key=...)

resp = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.0,      # ≈ deterministic
    messages=[
        {"role": "system",      # who you are, the rules
         "content": "You are a hiring manager..."},
        {"role": "user",        # this turn's input
         "content": "Which candidate...?"},
        {"role": "assistant",   # what the model said before
         "content": "Candidate3."},
    ],
)
resp.choices[0].message.content    # the reply text
```

- **Three roles:** system sets persona + rules; user is input; assistant is the model's own past output.
- **temperature 0** ≈ most reproducible
- **The API is stateless.**

mamab_llm.py at a glance



One environment, ten agents in a loop, one API call per agent per round — everything logged and checkpointed.

① An agent = a client + counts + a transcript

```
class Agent:
    def __init__(self, name, llm_client, num_arms,
                  communication):
        self.llm_client = llm_client    # its own OpenAI client

        # numeric bookkeeping (per arm)
        self.counts = np.zeros(num_arms)
        self.rewards = np.zeros(num_arms)
        self.alpha = np.ones(num_arms)    # Beta params
        self.beta = np.ones(num_arms)

        # what the LLM will "remember"
        self.llm_prompt = []    # the growing transcript
        self.llm_choices = []    # (arm, reward) history
        self.reward_history = {arm: {"1":0, "0":0}, ...}
```

- **One OpenAI client per agent** — ten independent “participants.”
- **Two kinds of state:** numeric variables (counts, per-arm success/failure) and the transcript the LLM sees.
- **reward_history** is the private variable — the direct analog of cumulativeResults in the human game.

② The system prompt: persona + output contract

```
# select_arm(), round 0 only — the persona:
"You are one of several hiring managers tasked with
maximizing your firm's rewards by hiring one of 10
candidate groups labeled Candidate1-Candidate10.
Each candidate group is associated with a Bernoulli
distribution ... with unknown probabilities. ...

# social version adds:
you will receive a summary of the results from both
your own choices and the choices made by other firms.

Let's think step by step ... provide your final answer
within the tags <Answer>CANDIDATE</Answer> where
CANDIDATE is one of Candidate1-Candidate10."

self.llm_prompt.append({"role": "system", ...})
```

- **Persona:** a hiring manager — mirrors the human instructions.
- **The condition lives here too:** the social variant announces that shared results will arrive.
- **Output contract:** answers must come inside **<Answer>...</Answer>** — machine-parseable by design.
- **“Think step by step”** elicits reasoning before the tag — logged, and analyzable later.

③ Each round: feedback + summary → user message

```
# generate_summary(reward_history):
"Candidate1: 12 times, average reward: 0.75"
"Candidate2: 41 times, average reward: 0.93"
"Candidate3: 0 times" ...

# select_arm(), every round:
summary = generate_summary(
    global_reward_history if communication
    else self.reward_history)

user_content = (
    f"In the last round, you hired Candidate{...}
    and the reward feedback was {last_reward}.
    ... summarized as follows:\n{summary}\n
    Which candidate group will you choose next?
    You MUST ... <Answer>CANDIDATE</Answer> ...")

self.llm_prompt.append({"role": "user", ...})
```

- **Sufficient statistics, not raw logs:** per-candidate pull counts + average reward — compact and readable.
- **The condition is one ternary:** pooled global history (social) vs the agent's own (asocial) — the same switch as the human client.
- **Last action + last reward** are restated explicitly — immediate feedback, like the human result display.
- **The format demand is repeated** every round. Repetition keeps compliance high in long runs.

④ Query the model, parse the answer

```
def _query_llm(self):
    resp = self.llm_client.chat.completions.create(
        messages=self.llm_prompt, # the ENTIRE transcript
        model="gpt-4o", temperature=0.0)
    return resp.choices[0].message.content.strip()

def _parse_response(response):
    match = re.search(
        r"<Answer>\s*Candidate\s*(10|[1-9])\s*</Answer>",
        response, flags=re.IGNORECASE)
    if not match:
        # fallback:
        match = re.search(r"\bCandidate\s*(10|[1-9])\b", ...)
    if match:
        return int(match.group(1)) - 1
    raise ValueError(f"No valid candidate ...")
```

- **messages = the whole transcript.** Every round re-sends everything — the memory model, examined shortly.
- **Parse defensively:** strict tag first, bare “CandidateN” as fallback, tolerant of case, spaces, stray periods.
- **Failure raises.** A malformed answer is the LLM’s dropout

⑤ The environment loop

```
class BanditEnvironment:
    ARM_PROBS = [0.53, 0.95, 0.30, 0.10, 0.68,
                 0.40, 0.16, 0.85, 0.22, 0.12]
    # 'equal' condition → [0.9] * 10

    def run_simulation(self, log_file, state_file):
        for round_num in range(self.current_round, 200):
            for i, agent in enumerate(self.agents):
                arm, resp = agent.select_arm(round_num,
                                             self.communication,
                                             self.global_reward_history)
                reward = self.arms[arm].sample_reward()
                agent.update(arm, reward)
                round_log["responses"].append({..., resp})

            ...update global history... → next slide
            _append_log(log_file, round_log)
            _save_state(state_file, ..., round_num + 1)
```

- **The same truth as the human game:**
ARM_PROBS is identical to the probabilities table in Choice.jsx (0.10 – 0.95); 'equal' = flat 0.9, also identical.
- **The environment draws the reward** — here it IS server-side. The agent only sees the outcome.
- **Raw responses are logged** — the full reasoning text of every call, for later analysis.

⑥ The social condition: pooling

```
# after every round, the environment pools:
for arm, reward in zip(choices, rewards):
    self.global_reward_history[arm][str(reward)] += 1

if self.communication:
    shared_alpha = np.mean([a.alpha for a in agents], 0)
    shared_beta = np.mean([a.beta for a in agents], 0)
    for agent in self.agents:
        agent.share_information(shared_alpha, shared_beta)

# ...and in the social prompt, the summary reads from
# global_reward_history – everyone's outcomes pooled.
```

- **global_reward_history** = the group ledger.
The agent's reward_history = the private ledger.
Same two-ledger design as Part 1.
- **Asocial agents never see the pool**; social agents' every summary is built from it.

⑦ Seeding and surviving crashes

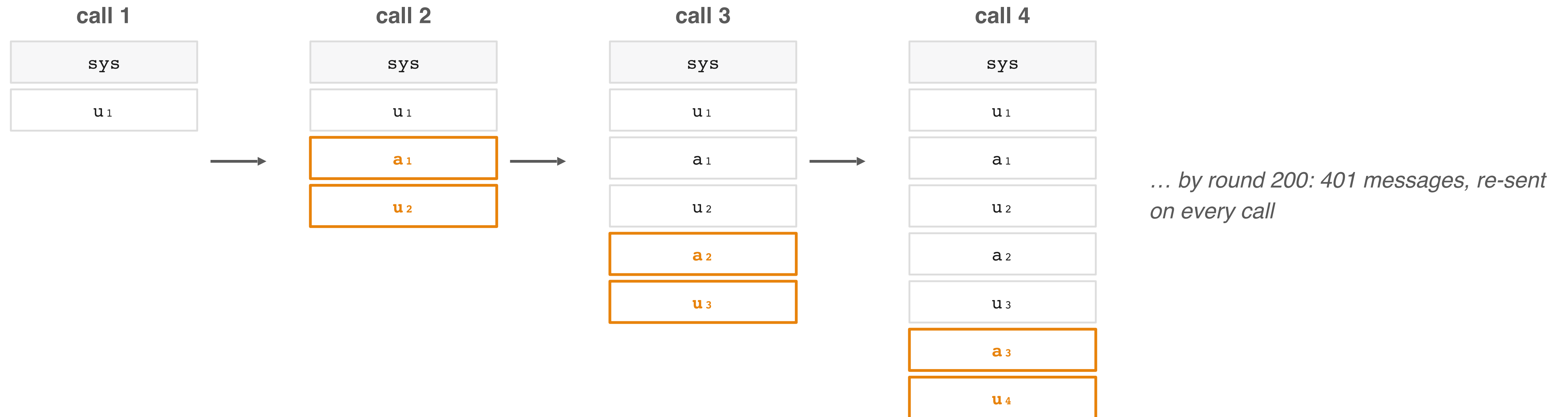
```
def _initialize_first_round(self, log_file):
    """Assign each agent to a unique arm on round 0."""
    for i, agent in enumerate(self.agents):
        agent.update(i, reward=1)    # agent i → arm i
        self.global_reward_history[i]["1"] += 1

    # every round:
    _save_state(state_file, agents,
                global_reward_history, round_num + 1)

    # on start:
    if os.path.exists(state_filename):
        agents, history, current_round = _load_state(...)
        # → the run resumes exactly where it stopped
```

- **Seeding.** Agent i starts on arm i with reward 1 — the tutorial round of the human game (10 players × 10 assigned arms), reborn as a loop.
- **Checkpoint every round.** The full state — including every transcript — serializes to JSON.
- **API runs fail:** rate limits, timeouts, overnight disconnects. Resume-from-checkpoint turns a crash into a pause.

How memory works here: replay everything



- **The transcript only grows:** system + (assistant, user) pair per round; the whole list is re-sent every call — the API is stateless.

Today's APIs: server-side conversation state

```
# Responses API – chain calls instead of replaying:
r1 = client.responses.create(
    model="gpt-5",
    instructions=SYSTEM_PROMPT,
    input="Round 1: ... which candidate?" )

r2 = client.responses.create(
    model="gpt-5",
    previous_response_id=r1.id,    # ← the chain
    input="Round 2: reward was 1. ...")

# only the NEW turn is sent; the server
# reconstructs the history behind r1.id
```

- **One id per agent replaces the transcript list**
— your code carries a pointer, not the history.
- **Cleaner code, simpler state files.** The checkpoint stores 10 ids instead of 10 transcripts.

Conversations — and the option to remember less

```
# Conversations API — a durable object per agent:  
conv = client.conversations.create() # once, per agent  
  
client.responses.create(  
    model="gpt-5",  
    conversation=conv.id,      # ← state rides on the id  
    input="Round 2: reward was 1. ...")  
  
# persists across sessions / machines / restarts —  
# your checkpoint is just {agent: conv_id}
```

- **conversation id > response chain** when runs resume across sessions — exactly our checkpointing use case.
- **The third option is a design choice:** remember LESS. Send system + current summary only — constant cost, and our summary is already sufficient.
- **Memory architecture is now an experimental variable:** full transcript vs summary-only can change agent behavior — measure it, don't assume it.

Thanks Q&A